

# Procesamiento del Lenguaje Natural

Redes Neuronales Recurrentes (RNN)

---

Jorge Ortiz Fuentes

6 de mayo de 2025

Introducción y motivación

Redes Neuronales Recurrentes (RNN)

Variantes de RNN

Arquitecturas con compuertas

LSTM (Long Short-Term Memory)

GRU (Gated Recurrent Unit)

Cierre y reflexión

# Introducción y motivación

---

# Consideraciones sobre el lenguaje

## El lenguaje es secuencial

- El lenguaje natural (texto, habla) es inherentemente **secuencial** y el significado de las palabras depende de su **contexto**.
- Los modelos *feed-forward* tradicionales procesan entradas de forma independiente, carecen de “memoria”.

## Necesidad de memoria

Requerimos modelos capaces de:

- Procesar secuencias de longitud variable.
- Mantener información de pasos anteriores (**memoria temporal**).
- Capturar dependencias entre elementos en la secuencia.

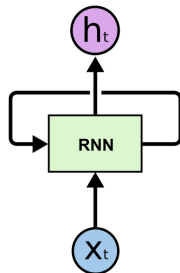
# Redes Neuronales Recurrentes (RNN)

---

# Abstracción de una RNN

## Idea central: recurrencia

- Una RNN procesa la secuencia paso a paso ( $t = 1, 2, \dots, T$ ).
- Mantiene un **estado oculto** ( $h_t$ ) que actúa como **memoria**.
- En cada paso  $t$ :
  - Recibe entrada  $x_t$ .
  - Recibe estado anterior  $h_{t-1}$ .
  - Calcula nuevo estado  $h_t$ .
  - (Opcional) Produce salida  $y_t$ .
- El estado  $h_t$  resume la información relevante de  $x_1, \dots, x_t$ .

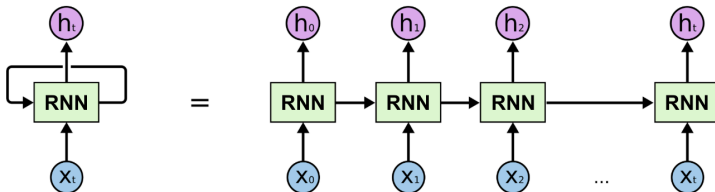


**Figura 1:** Celda RNN básica.

# Desenrollada en el tiempo

## Recurrencia

El bucle recurrente puede “desenrollarse” en el tiempo, revelando una cadena de procesamiento.



**Figura 2:** RNN desenrollada para una secuencia de longitud  $T$ .

## Importancia

**Crucial:** los mismos parámetros se usan en **cada paso temporal**.

- Reduce drásticamente el número de parámetros a aprender.
- Permite generalizar a secuencias de diferente longitud.
- El modelo aprende una transición *general* aplicable en cualquier punto de la secuencia.



# RNN simple (Elman Network)

## Arquitectura básica (Elman, 1990)

- Capa de entrada  $\mathbf{x}_t$ .
- Capa oculta recurrente  $\mathbf{h}_t$ .
- Capa de salida  $\mathbf{y}_t$ .

## Ecuaciones (Conceptuales)

$$\mathbf{h}_t = f(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = g(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)$$

Donde  $f, g$  son activaciones,  $\mathbf{W}$  pesos,  $\mathbf{b}$  sesgos.

## Formulas

$$\mathbf{h}_t = f(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\mathbf{y}_t = g(\mathbf{V}\mathbf{h}_t + \mathbf{b}_y)$$

- **W**: matriz de pesos de **entrada actual** ( $\mathbf{x}_t \rightarrow \mathbf{h}_t$ )
- **U**: matriz de pesos **recurrentes** ( $\mathbf{h}_{t-1} \rightarrow \mathbf{h}_t$ )
- **V**: matriz de pesos de **salida** ( $\mathbf{h}_t \rightarrow \mathbf{y}_t$ )
- $f$ : activación recurrente (típicamente tanh o ReLU)
- $g$ : activación de salida (depende de la tarea)

## Ventajas

- Conceptualmente simple.
- Capaz de procesar secuencias de longitud variable.
- Puede modelar dependencias temporales (en teoría).
- Menos parámetros que arquitecturas más complejas.

## Limitaciones

- **Problema del Gradiente Desvaneciente/Explosivo:**  
Dificultad para aprender dependencias a largo plazo.  
Gradientes se multiplican  $\rightarrow$  muy pequeños (vanishing) o muy grandes (exploding).
- La memoria es “frágil”: la información de pasos lejanos tiende a perderse o diluirse.
- Menos potente que arquitecturas más modernas (LSTM, GRU) para tareas complejas.

*Motivación para arquitecturas más sofisticadas.*

# Problema: gradientes desvanecientes / explosivos

## Gradiente Desvaneciente (Vanishing)

- Gradiente se reduce exponencialmente hacia atrás.
- Multiplicación repetida de valores  $< 1$ .
- **Consecuencia:** No aprende dependencias largas.

## Gradiente Explosivo (Exploding)

- Gradiente crece exponencialmente hacia atrás.
- Multiplicación repetida de valores  $> 1$ .
- **Consecuencia:** Entrenamiento inestable (NaNs).

*Principal obstáculo para las RNN simples.*

## Estrategias comunes

- **Inicialización de pesos cuidadosa:** Evitar empezar en regiones de saturación o magnitudes extremas (ej. Xavier, He, Ortogonal).
- **Funciones de activación alternativas:** ReLU y variantes (ej. Leaky ReLU) pueden mitigar el desvanecimiento.
- **Arquitecturas con Compuertas (LSTM, GRU):** La solución más robusta, controlan el flujo de información/gradientes. (Veremos a continuación)

## Variantes de RNN

---

## Motivación

En muchas tareas (ej. etiquetado, traducción), el contexto **futuro** es tan importante como el **pasado**.

- Ejemplo: Determinar si “banco” es *institución financiera* o *asiento* puede requerir ver palabras posteriores.
- Una RNN estándar solo ve el pasado ( $\mathbf{h}_t$  depende de  $t - 1, t - 2, \dots$ ).



# BiRNN: Arquitectura

## Arquitectura

- Dos RNNs independientes:
  - Una **hacia adelante** ( $\rightarrow$ ): captura  $\vec{h}_t$ .
  - Una **hacia atrás** ( $\leftarrow$ ): captura  $\overleftarrow{h}_t$ .
- Estados ocultos se **combinan** (ej. concatenan):
$$\mathbf{h}_t = [\vec{h}_t; \overleftarrow{h}_t]$$
- La salida  $\mathbf{y}_t$  se basa en  $\mathbf{h}_t$ .

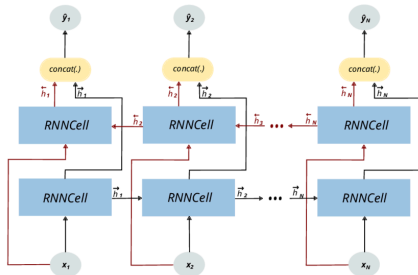


Figura 3: Arquitectura BiRNN.

## Ventajas

- Acceso a contexto completo (pasado y futuro) en cada punto.
- Generalmente mejora el rendimiento en tareas donde el contexto global es útil.

### Consideración importante

- Requiere procesar la secuencia **completa** antes de generar salidas basadas en contexto futuro.
- No es aplicable directamente a tareas de predicción *online* (en tiempo real) donde el futuro no está disponible al tomar la decisión en el tiempo  $t$ .

# RNN Multicapa (Stacked RNN): motivación

## Motivación

Al igual que en redes feed-forward, apilar capas puede permitir aprender representaciones más **abstractas** y **jerárquicas** de la secuencia.

- La capa 1 aprende patrones básicos.
- La capa 2 opera sobre las salidas de la capa 1, aprendiendo patrones de nivel superior.

## Arquitectura

- Se apilan múltiples capas RNN.
- Salida  $\mathbf{h}_t^{(l)}$  de capa  $l$  es entrada para capa  $l + 1$ :  
$$\mathbf{h}_t^{(l)} = f^{(l)}(\dots \mathbf{h}_t^{(l-1)} \dots \mathbf{h}_{t-1}^{(l)} \dots)$$
- Aumenta la “profundidad” en cada paso  $t$ .

## Consideraciones

**Beneficios:** Mayor capacidad expresiva.

**Desafíos:** Más parámetros, coste computacional, puede agravar problemas de gradiente.

# Arquitecturas con compuertas

---

## El problema de la memoria a largo plazo

Las RNN simples tienen dificultad para:

- **Retener** información relevante a largo plazo.
- **Descartar** información irrelevante.

Esto se debe en gran parte al problema del gradiente desvaneciente/explosivo que dificulta el aprendizaje de dependencias largas.

# La solución: mecanismos con compuertas (gates)

## Idea principal

- Introducir “compuertas” (gates) dentro de la celda RNN.
- Son redes neuronales pequeñas (sigmoide  $\rightarrow [0, 1]$ ) que **aprenden a controlar** el flujo de información.
- Deciden dinámicamente qué olvidar, qué incorporar y qué exponer.
- Objetivo: facilitar aprendizaje de dependencias largas mitigando problemas de gradiente.

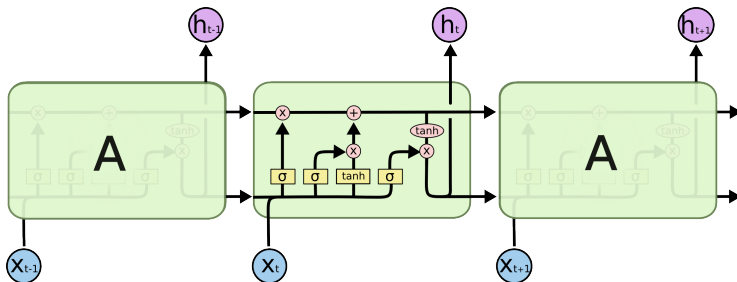
*Arquitecturas clave: LSTM y GRU.*



## Componentes principales (Hochreiter & Schmidhuber, 1997)

- **Estado de celda ( $c_t$ ):**
  - La “autopista” de información; memoria a largo plazo.
  - Modificaciones mínimas controladas por gates.
- **Estado oculto ( $h_t$ ):**
  - Salida de la celda en  $t$ ; memoria a corto plazo.
  - Basado en  $c_t$  filtrado.
- **Compuertas (gates):** regulan  $c_t$  y  $h_t$ .

# LSTM: conceptos clave



**Figura 4:** Diagrama general de una cadena LSTM.

## Flujo de información

La imagen muestra cómo la información ( $x_t$ ), el estado oculto ( $h_{t-1}$ ) y el estado de celda ( $c_{t-1}$ ) fluyen a través de las compuertas para generar las nuevas versiones ( $h_t$ ,  $c_t$ ).

## 1. Compuerta de olvido ( $f_t$ )

Decide qué información desechar del estado de celda anterior  $c_{t-1}$ . Basada en  $h_{t-1}$  y  $x_t$ .

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

( $\sigma$ : sigmoide.  $f_t \in [0, 1]$ . 1=mantener, 0=olvidar)

## 2. Compuerta de entrada ( $i_t, \tilde{c}_t$ )

Decide qué nueva información almacenar en  $c_t$ .

- *Qué valores actualizar* ( $i_t \in [0, 1]$ ):

$$i_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

- *Valores candidatos a añadir* ( $\tilde{c}_t \in [-1, 1]$ ):

$$\tilde{c}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

Luego, actualiza estado de celda:  $\mathbf{c}_t = \mathbf{f}_t * \mathbf{c}_{t-1} + i_t * \tilde{c}_t$  (**Adición!**)

## 3. Compuerta de salida ( $\mathbf{o}_t$ )

Decide qué partes del estado de celda  $\mathbf{c}_t$  exponer.

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

## Estado oculto ( $\mathbf{h}_t$ )

Calcula el estado oculto (salida de la celda).

$$\mathbf{h}_t = \mathbf{o}_t * \tanh(\mathbf{c}_t)$$

(\*: multiplicación elemento a elemento)

# LSTM: ¿cómo resuelve el vanishing gradient?

## El rol clave del estado de celda ( $c_t$ )

- La operación **aditiva** ( $c_t = \dots + \dots$ ) es crucial. Permite que la información (y gradientes) fluyan más fácilmente.
- Si  $f_t \approx 1$  y  $i_t \approx 0$ , entonces  $c_t \approx c_{t-1}$ . La información se preserva casi intacta  $\rightarrow$  “autopista” para el gradiente.
- Evita la multiplicación repetida de términos pequeños que causa el desvanecimiento en RNN simples.
- Las compuertas **aprenden** cuándo preservar y cuándo actualizar la memoria.

# GRU: una alternativa simplificada

## Concepto (Cho et al., 2014)

- Simplifica la arquitectura LSTM.
- Combina estado de celda y oculto en  $h_t$ .
- Solo **dos compuertas**:
  - Actualización ( $z_t$ ).
  - Reinicio ( $r_t$ ).
- Menos parámetros, más eficiente.

## GRU: diagrama de la celda

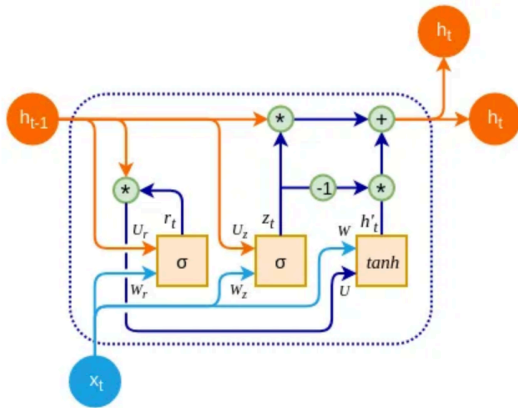


Figura 5: Diagrama detallado de una celda GRU.



## 1. Compuerta de reinicio ( $r_t$ )

Decide cuánto del estado anterior  $\mathbf{h}_{t-1}$  “olvidar” al calcular la info candidata  $\tilde{\mathbf{h}}_t$ .

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

Calcula el estado candidato  $\tilde{\mathbf{h}}_t$ :

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t * \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

( $\mathbf{r}_t \approx 0 \implies$  ignora  $\mathbf{h}_{t-1}$  para el candidato).

# GRU: compuerta de actualización y estado final

## 2. Compuerta de actualización ( $z_t$ )

Decide cuánto mantener de  $\mathbf{h}_{t-1}$  vs. cuánto incorporar de  $\tilde{\mathbf{h}}_t$ .

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z)$$

## Estado final ( $\mathbf{h}_t$ )

Calcula el estado final (interpolación):

$$\mathbf{h}_t = (1 - \mathbf{z}_t) * \mathbf{h}_{t-1} + \mathbf{z}_t * \tilde{\mathbf{h}}_t$$

*Estructura aditiva ayuda a mitigar vanishing gradient.*

## Cierre y reflexión

---

# ¿Cuándo elegir cada modelo?

## Guía general

- **RNN Simple (Elman):**
  - *Cuándo:* Rara vez hoy día. Quizás para secuencias muy cortas, problemas muy simples, fines educativos.
  - *Por qué:* Sufre mucho con dependencias largas (vanishing gradient).
- **LSTM:**
  - *Cuándo:* Tareas con dependencias potencialmente muy largas, robustez clave, recursos computacionales suficientes. Era el “caballo de batalla”.
  - *Por qué:* Buen manejo de memoria larga, arquitectura probada.

# ¿Cuándo elegir cada modelo?

## Guía general

- **GRU:**

- *Cuándo:* Similar a LSTM, pero se busca mayor eficiencia (rapidez, menos parámetros), o datos limitados. Buena primera opción a probar.
- *Por qué:* Equilibrio rendimiento/eficiencia.

- **BiRNN (BiLSTM/BiGRU):**

- *Cuándo:* Tareas *offline* (secuencia completa disponible) donde contexto futuro ayuda (etiquetado, codificación seq2seq).

# ¿Cuándo elegir cada modelo?

## Guía general

- **Stacked RNN (Stacked LSTM/GRU):**
  - *Cuándo:* Modelar jerarquías complejas, si más profundidad mejora rendimiento (con suficiente data/regularización).

*Nota: Los Transformers ahora dominan muchas de estas tareas.*

## La era de los Transformers

- Arquitecturas basadas en **mecanismos de atención** (ej. Transformer) han superado a las RNNs en muchas tareas SOTA de NLP desde 2017 (Attention is All You Need).
- **Ventajas clave de Transformers:**
  - Mejor manejo de dependencias muy largas (acceso directo, no secuencial).
  - Mayor paralelización (entrenamiento más rápido en hardware adecuado).

## ¿Están obsoletas las RNNs?

### ¡No! Siguen siendo relevantes

- **Procesamiento online/streaming:** Donde la secuencia llega paso a paso y el futuro no se conoce.
- **Eficiencia:** Pueden ser más ligeras (menos parámetros/cómputo) que Transformers grandes, útil en recursos limitados.
- **Tareas específicas:** Donde la naturaleza secuencial inherente es ventajosa o suficiente.
- **Bloques constructivos:** Usadas en arquitecturas híbridas.
- **Fundamento conceptual:** Comprender RNNs/LSTM/GRU es clave para entender la evolución del campo.



## Puntos clave

- El NLP requiere modelos que manejen **secuencias y memoria**.
- Las **RNNs** introducen recurrencia ( $\mathbf{h}_t$  depende de  $\mathbf{h}_{t-1}$ ), pero las simples sufren de **problemas de gradiente** (vanishing/exploding).

## Puntos clave

- **BiRNNs** usan contexto pasado y futuro; **Stacked RNNs** aumentan la profundidad.
- **LSTM** y **GRU** introducen **compuertas** para controlar flujo de información/gradientes, modelando eficazmente dependencias largas.
- La elección del modelo (RNN simple, LSTM, GRU, Bi, Stacked) depende de la tarea, datos, dependencias y recursos.

## Puntos clave

- Aunque los **Transformers** dominan actualmente muchas tareas SOTA, las RNNs siguen siendo relevantes y conceptualmente fundacionales.

-  Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8), 1735-1780.
-  Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv:1406.1078*.
-  Elman, J. L. (1990). Finding structure in time. *Cognitive science*, 14(2), 179-211.
-  Vaswani, A., et al. (2017). Attention is all you need. *NIPS*.